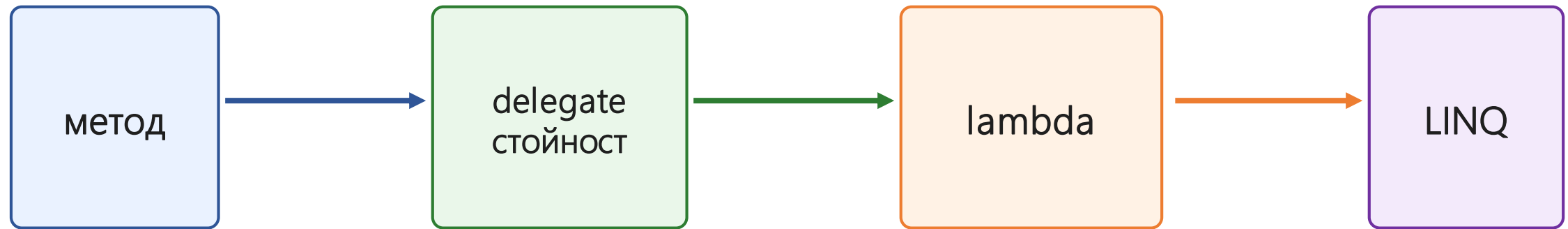


Функции като стойности

Делегати, lambda изрази и функции от по-висок ред в C#

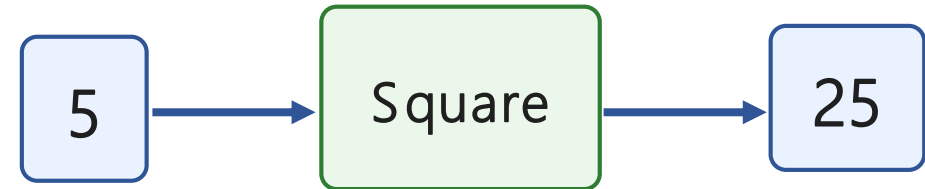


Може ли поведението да бъде подадено като данни?

Чиста функция: явен вход и резултат, без наблюдаем страничен ефект

Познат пример

```
static int Square(int x)
{
    return x * x;
}
```



Можем ли да запишем поведението на `Square` в променлива?

Функцията като стойност

В C# това се реализира чрез delegate типове

Познато: данни в променливи

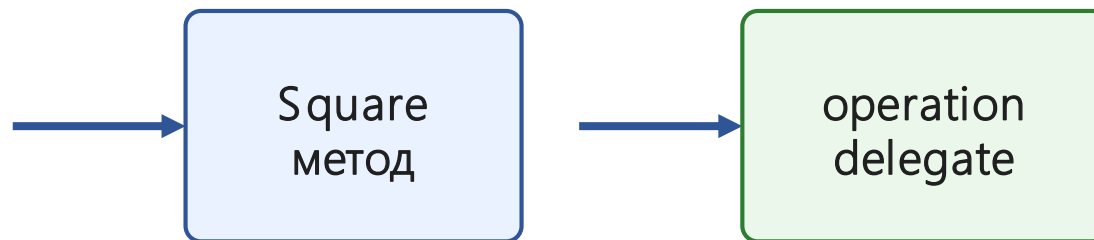
```
int number = 5;  
string name = "Ivan";
```

Новото: поведение в променлива

```
Func<int, int> operation = Square;
```

Извикване чрез променливата

```
int result = operation(5);  
Console.WriteLine(result); // 25
```



Delegate = представител на метод / поведение

Метод $\neq$ delegate

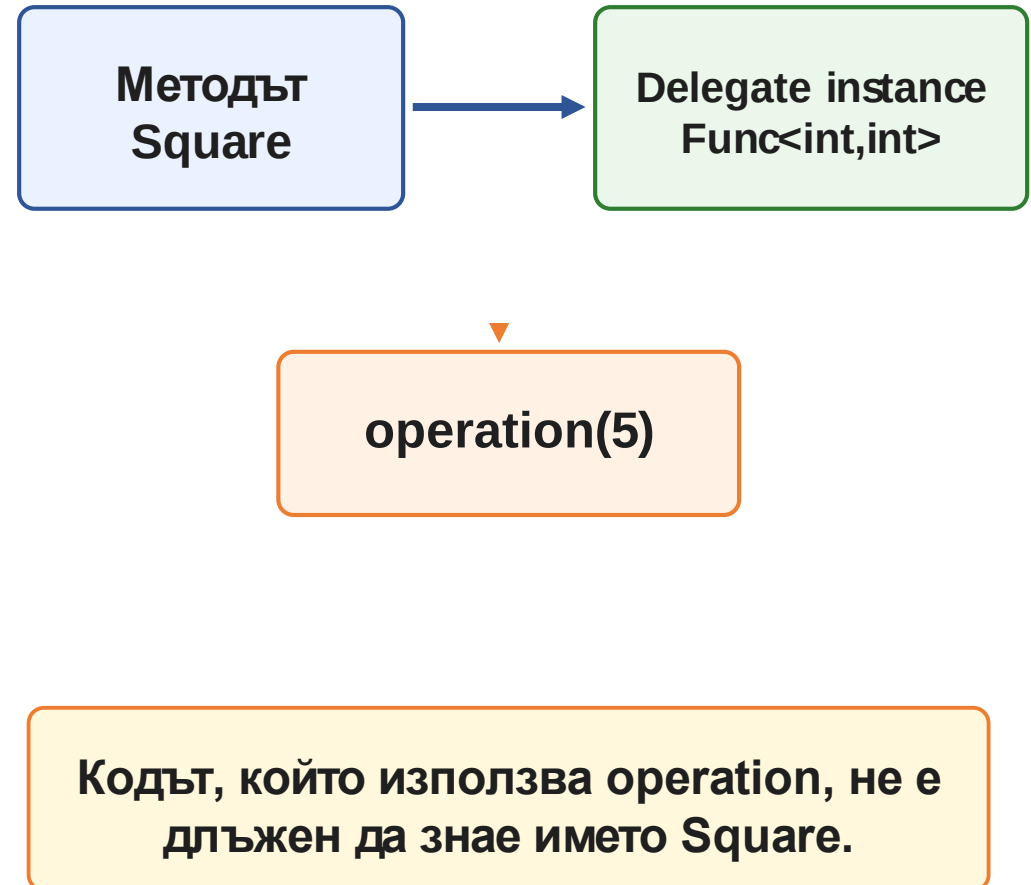
Методът е код. Делегатът е стойност, чрез която може да се извика метод.

Пример

```
static int Square(int x)
{
    return x * x;
}

Func<int, int> operation = Square;

int result = operation(5);
```



Пов едението може да се сменя

Една променлива може да представлява различно поведение

Два метода

```
static int Add(int a, int b)
{
    return a + b;
}

static int Multiply(int a, int b)
{
    return a * b;
}
```

Една delegate променлива

```
Func<int, int, int> operation;

operation = Add;
Console.WriteLine(operation(4, 5)); // 9

operation = Multiply;
Console.WriteLine(operation(4, 5)); // 20
```

Не променяме данните. Променяме функцията, която ще се изпълни.

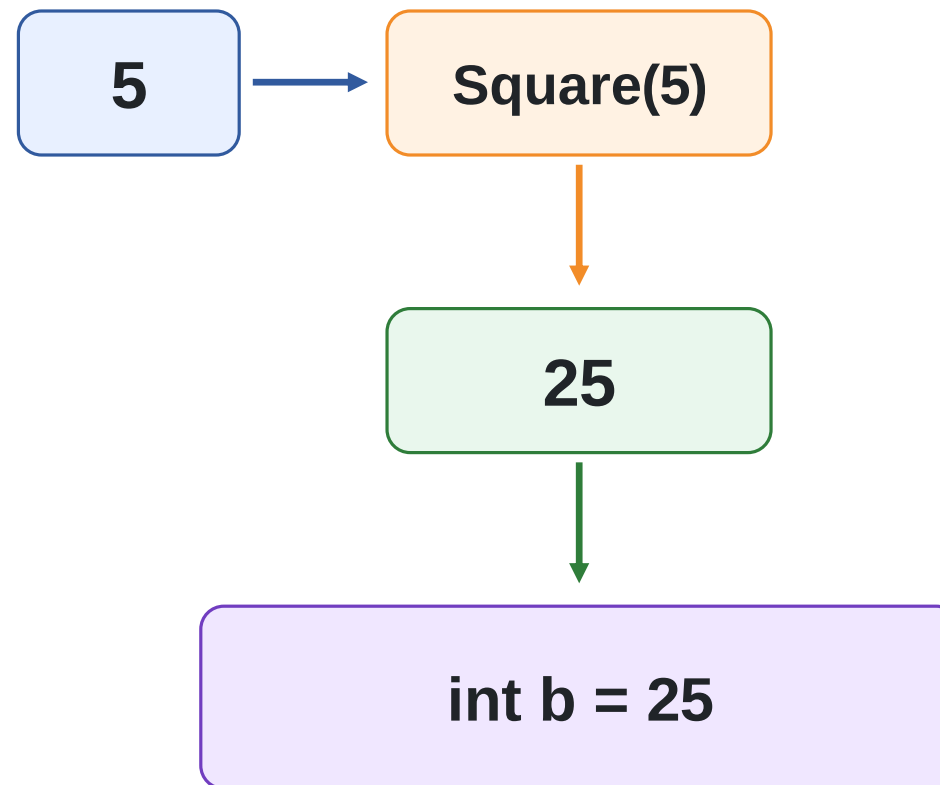
Държа стойност: резултатът вече е изчислен

Методът се извиква веднага, а в променливата се пази стойност.

```
static int Square(int x)
{
    return x * x;
}

int b = Square(5);
```

**В b НЕ се пази методът.
В b се пази само резултатът от
извикването.**



Изчисли сега и запази резултата.

Държа поведение: методът може да се извика по-късно

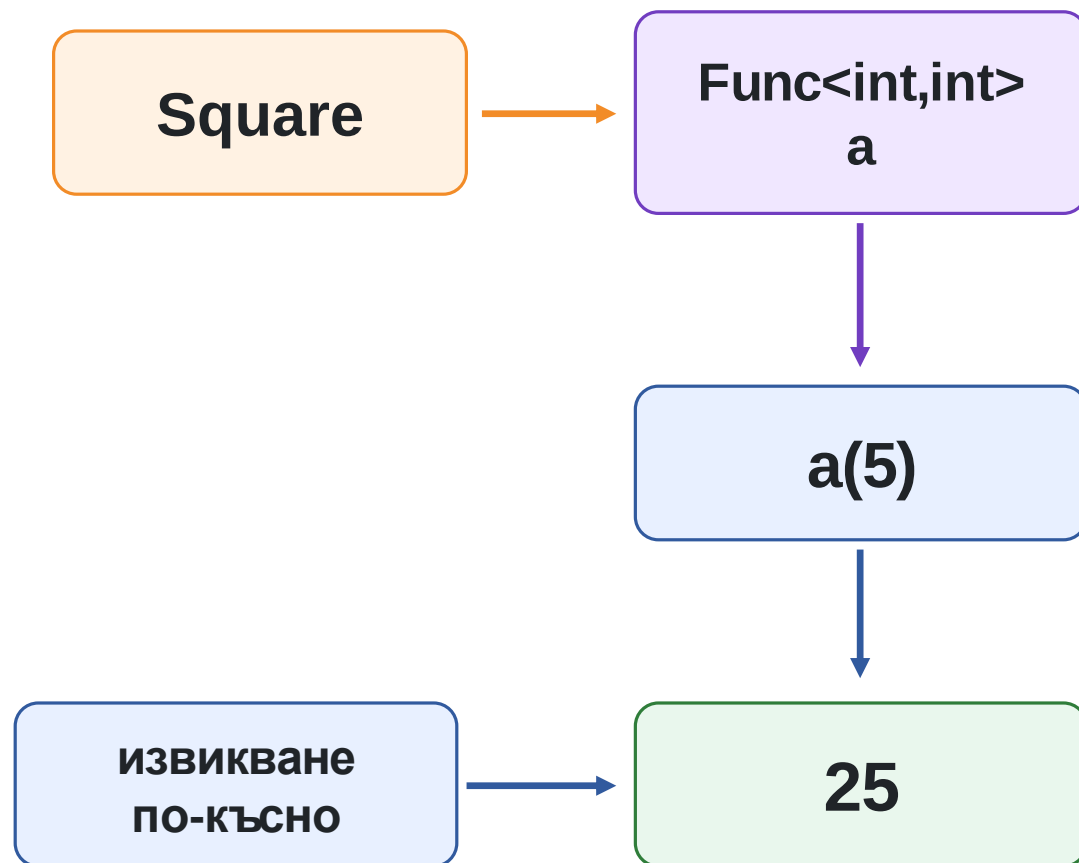
Делегатът представя метод/поведение като стойност.

```
static int Square(int x)
{
    return x * x;
}

Func<int, int> a = Square;
```

```
int result = a(5);
```

**Запази какво да се прави, а не готовия
резултат.**



Това е стъпка към „функция като стойност“.

Стойност срещу поведение

Една буква може да промени смисъла на целия ред код.

ДЪРЖА СТОЙНОСТ

```
int b = Square(5);
```

Square се извиква веднага

b съдържа 25

b може да участва в изрази или стойността ѝ да се изпечата

ДЪРЖА ПОВЕДЕНИЕ

```
Func<int,int> a = Square;
```

Square не се извиква веднага

a съдържа делегат към поведение

a може да бъде извикан: a(5)

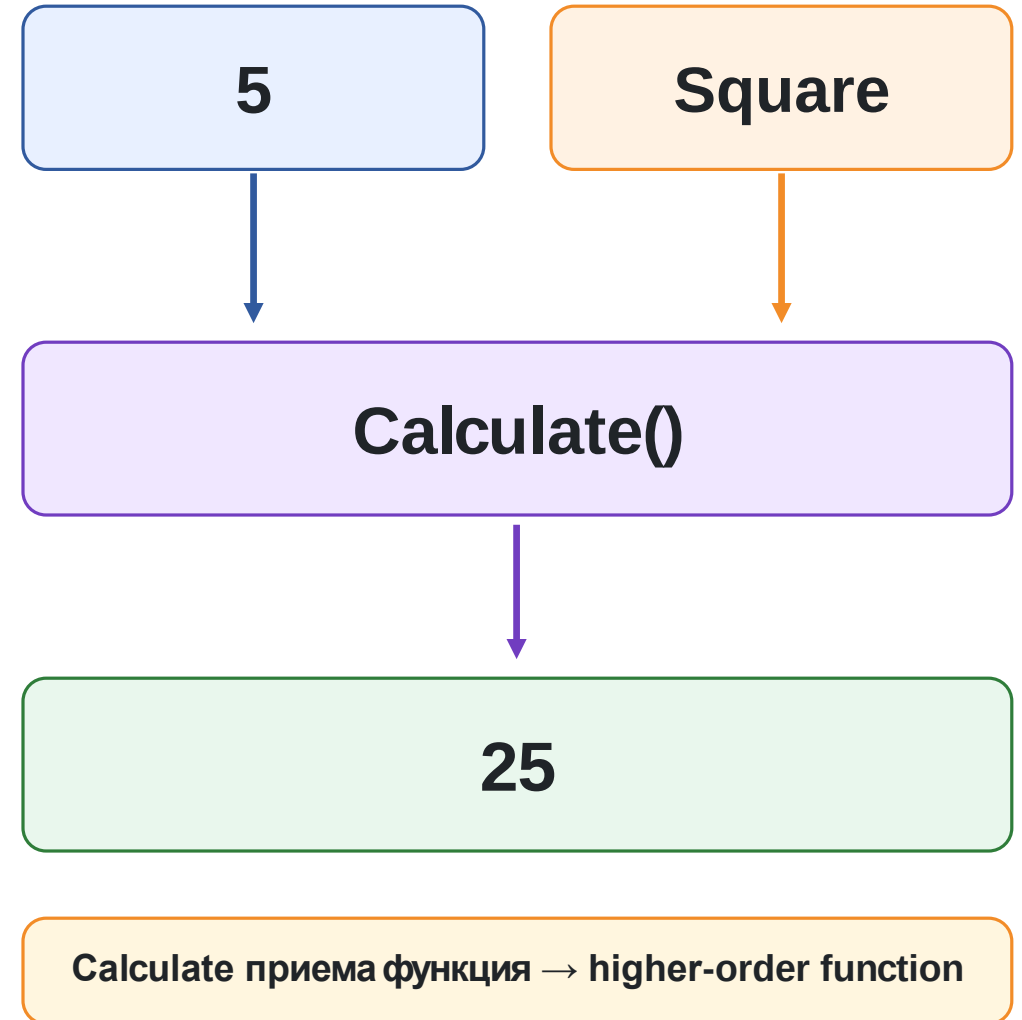
Delegate = механизъм в C#, чрез който поведение се използва като стойност.

От делегат към функция от по-висок ред

Когато метод приема поведение като параметър, той става функция от по-висок ред.

```
static int Calculate(  
    int x,  
    Func<int, int> operation)  
{  
    return operation(x);  
}
```

```
int result = Calculate(5, Square);
```



First-class function и higher-order function

Две свързани, но различни понятия

First-class function

функция може да се запише в променлива
функция може да се подаде като аргумент
функция може да се върне като резултат

C# чрез delegate

```
Func<int, int> f = Square;
```

Higher-order function

функция приема друга функция
и/или връща функция като резултат

Пример

```
static int Calculate(  
    int x,  
    Func<int, int> operation)
```

В C#. методите не са обекти, но могат да бъдат представени чрез delegate.

Функция като параметър

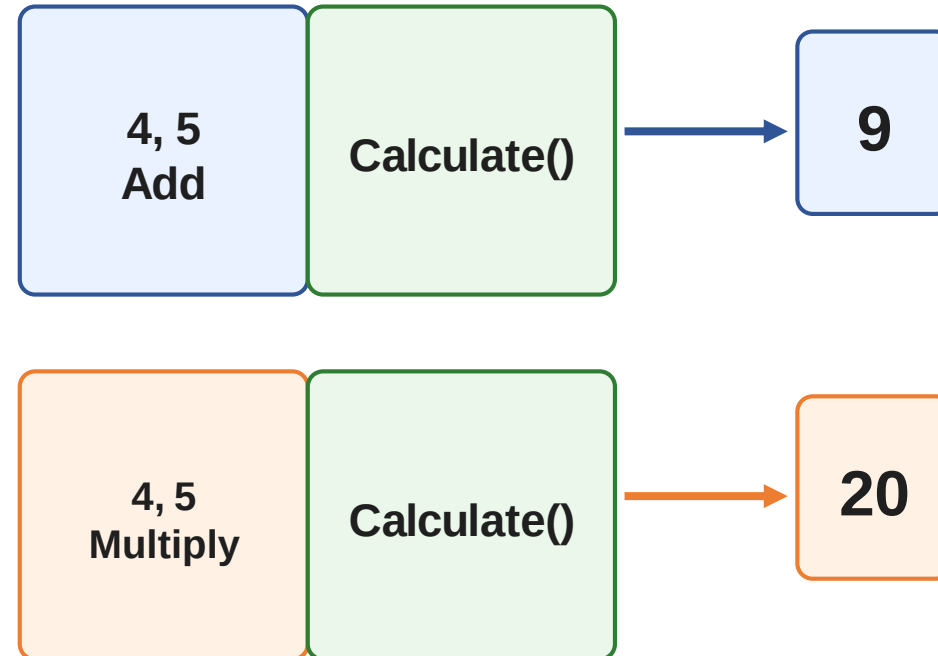
higher-order function

Функция приема друга функция

```
static int Calculate(  
    int a,  
    int b,  
    Func<int, int, int> operation)  
{  
    return operation(a, b);  
}
```

Извикване

```
int result1 = Calculate(4, 5, Add);  
int result2 = Calculate(4, 5, Multiply);
```



Lambda expression

Кратък синтаксис за анонимна функция

Именуван метод

```
static int Square(int x)
{
    return x * x;
}
```

Lambda

```
Func<int, int> square = x => x * x;
```

x

параметър

=>

lambda
оператор

x * x

израз / резултат

**x => x * x означава:
„приеми x и върни x * x“**

Func, Predicate, Action

Готови delegate типове, които се използват много често

Delegate	Получава	Връща	Пример
Func<T,TResult>	T	TResult	Func<int,int> square
Predicate<T>	T	bool	Predicate<int> isEven
Action<T>	T	void	Action<string> print

Код

```
Func<int, int> square = x => x * x;  
Predicate<int> isEven = x => x % 2 == 0;  
Action<string> print = text => Console.WriteLine(text);
```

Чисто поведение и страничен ефект

Delegate може да представя както чиста функция, така и действие със страничен ефект

Чиста функция

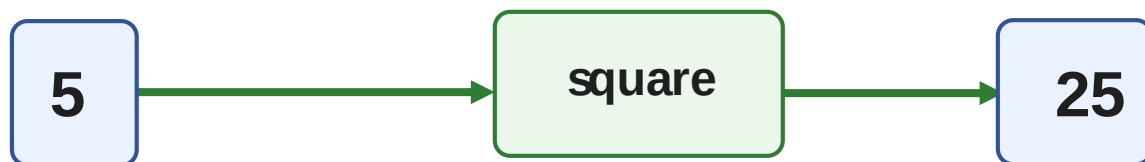
```
Func<int, int> square = x => x * x;
```

един и същ вход → един и същ резултат
без Console, без файл, без промяна на обект

Страничен ефект

```
Action<string> print = text =>  
Console.WriteLine(text);
```

не връща резултат
пише във външния свят: конзолата



Собствен Filter без LINQ

Алгоритъмът е един и същ, условието се подава като функция

Filter

```
static List<int> Filter(
    int[] numbers,
    Func<int, bool> condition)
{
    List<int> result = new List<int>();

    foreach (int number in numbers)
    {
        if (condition(number))
        {
            result.Add(number);
        }
    }

    return result;
}
```

Използване

```
int[] numbers = { 2, 5, 8, 11, 14 };

var even = Filter(numbers, x => x % 2 == 0);

var large = Filter(numbers, x => x > 10);
```

condition(number) е мястото, където Filter извиква подадената функция.

Това е функция от по-висок ред.

Връзката с LINQ: Where

Where прави същата идея върху `IE enumerable<T>`

По-разгънато

```
Func<int, bool> isEven = x => x % 2 == 0;  
var result = numbers.Where(isEven);
```



Обичайният запис

```
var result = numbers  
    .Where(x => x % 2 == 0);
```

predicate
`x => x % 2 == 0`



Where не „знае“ предварително условието. Получава го като функция.

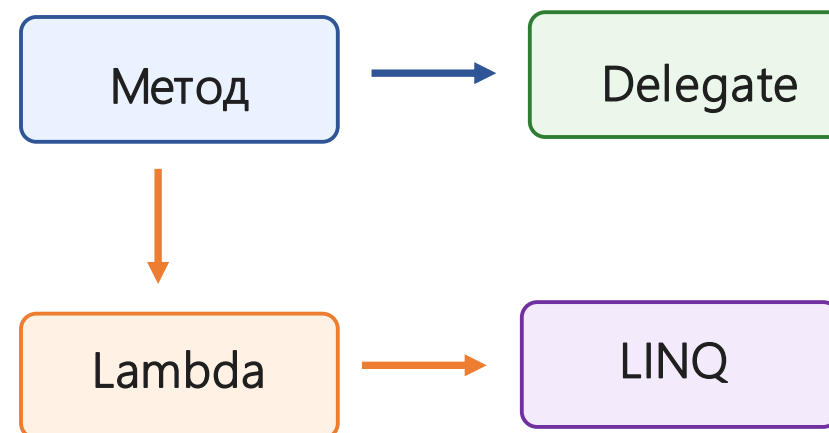
Какво да запомним?

Delegate представя метод/поведение като стойност.

В C# first-class functions се реализират основно чрез delegate типове.

Higher-order function приема или връща функция.
Lambda expression е кратък начин да запишем анонимна функция.

Where получава predicate функция, която връща true/false.



Крайна връзка

```
var result = numbers.Where(x => x % 2 == 0);
```

Въпроси за проверка

- 1 Каква е разликата между метод и delegate?
- 2 Какво е first-class functions?
- 3 Какво е higher-order function ?
- 4 Какво е lambda?
- 5 Какво е разликата между `Func<int, int>` и `Func<int, int, int>`?